Parallel Scaling

Ana Costa Pereira, Joshua Davies

26/06/2026

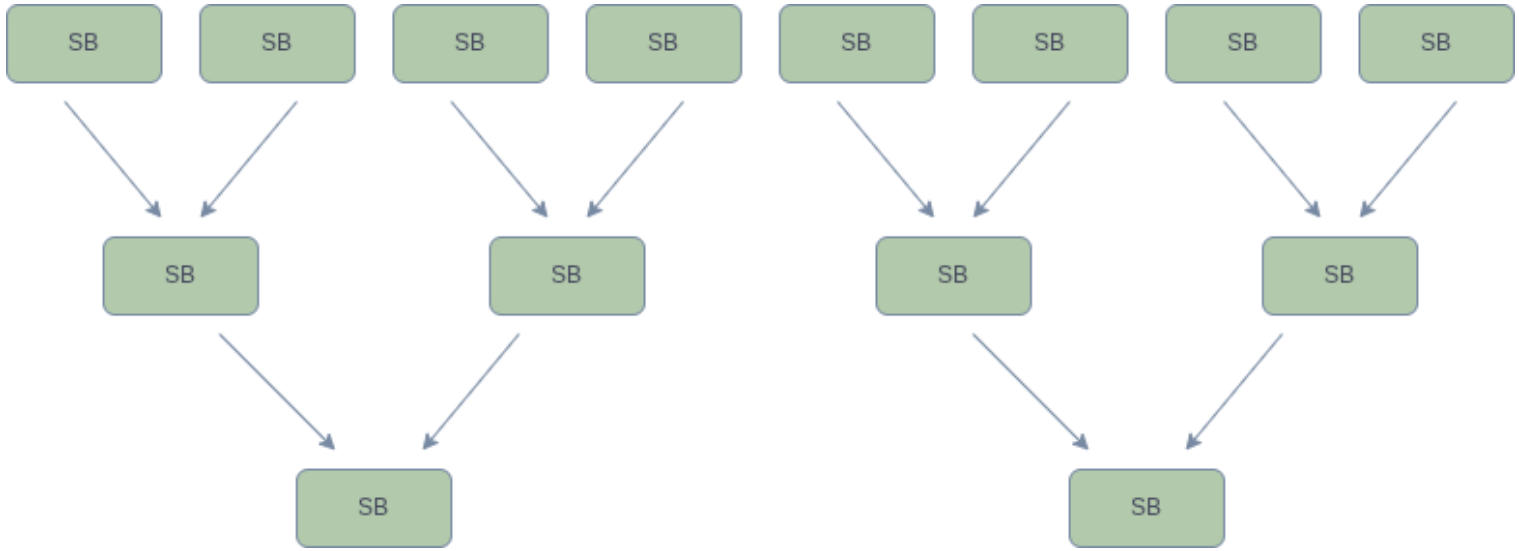
1. Parallel scalings

- Parallel scalings: study of scaling at higher thread number of tform;

Outline of talk

- Overview of sorting process in parallel;
- Benchmarks performenca at different thread count;
- Changes implemented in sorting of terms;

Parallel Sorting in TForm



- Master makes the allocations for the workers;
- For each N workers, $N - 2$ SB are created;

Parallel Sorting in TForm

- Each worker performs a local sort of its generated terms.
- Workers produce independent sorted streams.
- Sortbots merge pairs of sorted outputs in parallel.
- The master performs only the final merge and writes output.
- Goal: reduce master-thread bottlenecks and maximize worker utilization.

Sorting Resources

- Memory is shared globally between all threads;
- Avoid overwritting: each thread makes a private copy od the date: allocation of 1/N of the master's buffer size
- Fast in-memory sorting starts in the buffers.
- When buffers fill up, data is spilled to disk.

Working of the code:

- Terms are allowed to overlap between blocks;
- Reading thread always locks pair of locks - the current thread and the previous thread;
- 0th block - the overlap in the end of term is copied to 0th so that the terms are contiguous - this is no longer needed when we impose that only full terms can fit into the blocks

Before changes

- Default buffer sizes were increased over time
- Size of sortbots increased;
- The output from a worker fits in a single block $\longrightarrow$ no advantage on running in parallel;
- Each stage of sortbot merge tree runs one after the other
- Terms can be partially shared between blocks: when a term is being written in a block and the block finishes, the term keeps being written into the next block

After changes

- Blocks are made smaller
- Whole terms can only be written in one block: if it does not fit, it goes into the next block
- Workers that fill the blocks try to detect if there is a reader waiting for them
- Obtain a lock of the block before filling the block; if not, it means the reading thread already has the lock which means that it is processing the block which means it wants small terms from the current filling block, so we move on to the next block in writing thread
- There is always a minimum number of terms in the block - MINIMUMNUMBEROFTERMS

Some results after the changes

- No decrease in performance for any tests;
- Performance improvement for benchmarks where term merging is expensive (PolyRatFun, PolyRatFunExpand)
- Forcer, mbox1l